```
/**
 * server/lib/youtube-keyword-sanitizer.ts
 *
 * Fix #1 — Invalid Keywords Causing Permanent Upload Failures
 *
 * Sanitizes tags/keywords before any YouTube API call (videos.insert,
 * videos.update, thumbnails.set). The invalid_keywords error causes
 * permanent failures in the push-backlog — this stops that.
 *
 * YouTube keyword rules enforced here:
 *   - Each tag: 1–100 characters
 *   - Total across all tags: ≤500 characters
 *   - No HTML-like characters (< >)
 *   - No leading/trailing whitespace
 *   - No empty tags
 *   - Tags that are comma-joined strings get split into individual tags
 *   - Max 500 tags (safety cap — YouTube's actual limit varies)
 */

import { createLogger } from './logger.js';

const log = createLogger('youtube-keyword-sanitizer');

const MAX_TAG_CHARS   = 100;
const MAX_TOTAL_CHARS = 500;
const PROHIBITED_CHARS = /[<>"'\\]/g;

export function sanitizeYouTubeTags(rawTags: string[]): string[] {
  if (!Array.isArray(rawTags) || rawTags.length === 0) return [];

  const expanded = rawTags
    .flatMap(t => String(t ?? '').split(','))   // split any comma-joined tags
    .map(t => t
      .trim()
      .replace(PROHIBITED_CHARS, '')            // strip prohibited chars
      .replace(/\s+/g, ' ')                     // collapse whitespace
    )
    .filter(t => t.length > 0 && t.length <= MAX_TAG_CHARS);

  // Enforce total character limit
  let totalChars = 0;
  const sanitized: string[] = [];

  for (const tag of expanded) {
    const cost = tag.length + (sanitized.length > 0 ? 1 : 0); // +1 for comma separator
    if (totalChars + cost > MAX_TOTAL_CHARS) break;
    sanitized.push(tag);
    totalChars += cost;
```

```
  }

  if (sanitized.length < rawTags.length) {
    log.debug(
      `[KeywordSanitizer] Trimmed ${rawTags.length - sanitized.length} tags ` +
      `(${rawTags.length} → ${sanitized.length}) to fit YouTube limits`
    );
  }

  return sanitized;
}

/**
 * Validates a single tag and returns a human-readable reason if invalid.
 * Use in tests or for logging which specific tag caused the rejection.
 */
export function validateTag(tag: string): { valid: boolean; reason?: string } {
  if (!tag || tag.trim().length === 0) return { valid: false, reason: 'empty' };
  if (tag.length > MAX_TAG_CHARS)      return { valid: false, reason: `exceeds ${MAX_TAG_CHARS}
chars` };
  if (PROHIBITED_CHARS.test(tag))      return { valid: false, reason: 'contains prohibited
characters' };
  return { valid: true };
}
```